UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 04

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: JavaScript C TypeScript, Manipulación del DOM, Funciones Avanzadas (Closures, IIFE, Arrow), Canvas API y Animaciones
Modalidad: Presencial / Laboratorio
Fecha: 23.009.2026
Duración: 80 minutos

APELLIDOS Y NOMBRES: _____

OBJETIVO DE LA PRÁCTICA

Manipular el Árbol DOM dinámicamente con JavaScript moderno (ES6+), aplicar estructuras de control, funciones flecha, autoinvocadas (IIFE) y closures para gestión de estado, e implementar una animación interactiva usando Canvas API. Comprender el motor de ejecución JS, el Event Loop, el ciclo de renderizado y estrategias de optimización (requestAnimationFrame, prevención de fugas de memoria).

RECURSOS REQUERIDOS

- Visual Studio Code (Extensiones: Live Server, JavaScript/TypeScript Language Features, ESLint, Canvas Preview)
- Navegador Chrome/Firefox + DevTools (Consola, Elements, Performance, Memory)
- TypeScript (opcional: tsc global o playground online para validación de tipos)
- Documentación oficial: MDN Web Docs (DOM, Canvas, requestAnimationFrame)
- Conexión a internet

PARTE 1: EJERCICIO PASO A PASO

Paso	Actividad	Concepto Clave	Restricción/Nota Técnica
1	Crear index.html, app.js (o app.ts), estructurar UI con controles y <canvas>	Integración JS/TS, carga diferida (defer), estructura DOM	No usar frameworks ni librerías externas.

			Vanilla JS/TS únicamente
2	Implementar IIFE para aislamiento de scope, closures para estado de animación y arrow functions para handlers	Execution context, closure scope, this lexical binding	Documentar en comentarios cómo el closure retiene el estado entre frames
3	Manipulación DOM: querySelector, addEventListener, modificación dinámica de estilos/clases, validación de inputs	DOM API, Event Bubbling/Capturing, reflow vs repaint	Evitar style inline. Usar classList.toggle() y variables CSS para rendimiento
4	Canvas: contexto 2D, requestAnimationFrame, loop de renderizado, dibujo básico (arc, fillRect, stroke)	Event Loop, render pipeline, FPS, sincronización con display refresh	Prohibido setInterval para animación. Usar delta time (dt) para movimiento uniforme
5	Depuración y optimización: Console Profiler, Memory Snapshot, detección de listeners huérfanos, FPS counter	Garbage collection, detached nodes, cancelAnimationFrame, performance metrics	Registrar métricas en README.md. IA solo para interpretar gráficos de Performance

USO DE HERRAMIENTAS DE IA (Recomendado para optimizar el ejercicio)

Herramienta	Uso Permitido	Prompt Ejemplo	Uso Prohibido
ChatGPT/Claude/Copilot	Explicar closure vs global scope, depurar context null, optimizar	"¿Por qué mi closure pierde	"Genera el código completo

	requestAnimationFrame	referencia actxtras 100 frames? Dame patrón de encapsulación seguro"	de una animación de partículas con canvas y controles DOM"
TypeScript Playground	Validar tipos, inferencia de funciones, interfaces para state	(Oficial)	Usar any o as any para silenciar errores sin justificación técnica
DevTools Performance	Medir FPS, detectar layout thrashing, analizar heap	(Automático)	Ignorar advertencias de Detached DOM nodes o Long Tasks

Regla de laboratorio: El 70% de la lógica de animación y manipulación DOM debe escribirse manualmente. Cada uso de IA debe llevar comentario

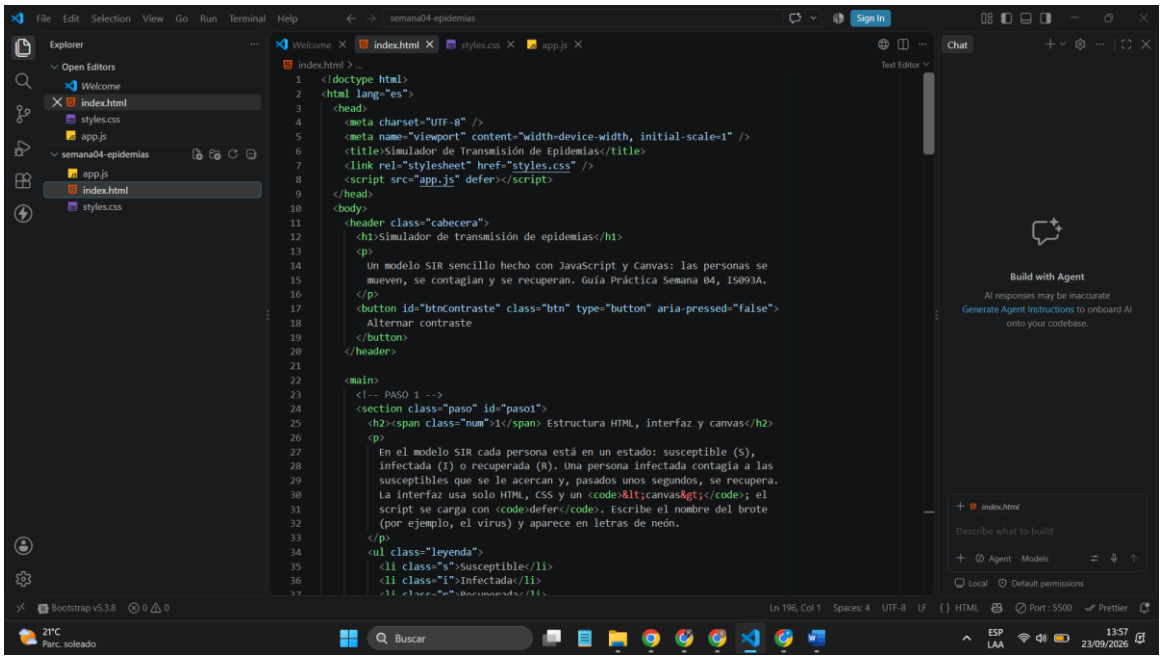
Evidencias del Ejercicio Desarrollado:

- Añada capturas de pantalla de cada paso realizado en Visual Studio Code.
- Añada capturas de la Página Web implementada de manera local
- Proporcione el enlace de Github de su proyecto

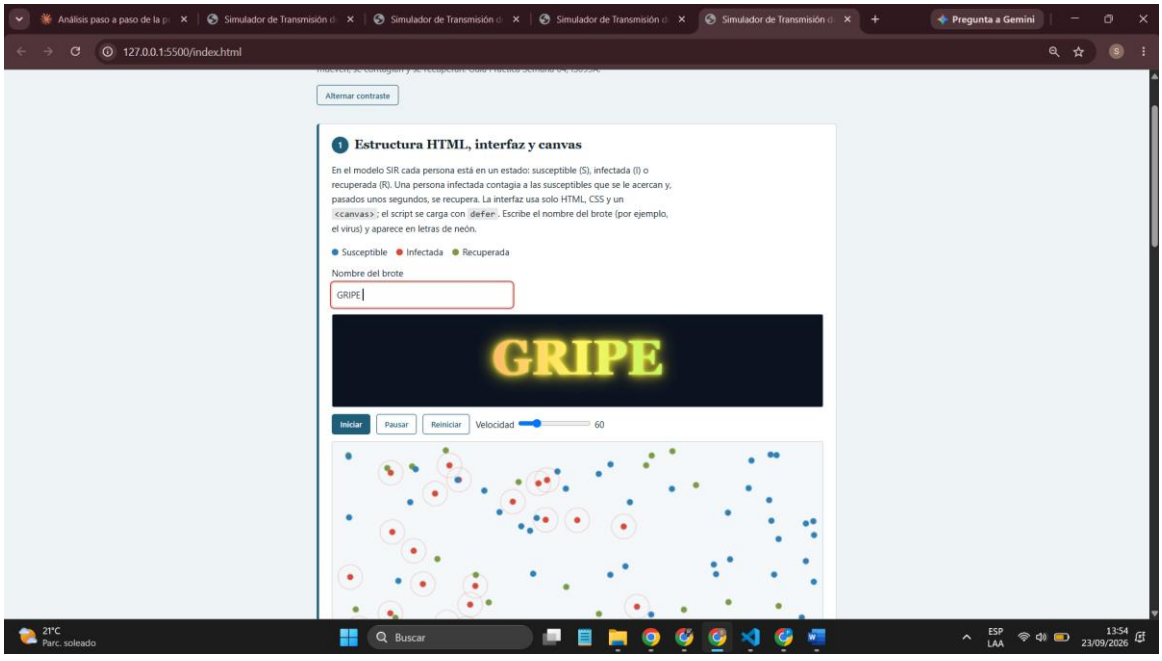
EVIDENCIAS:

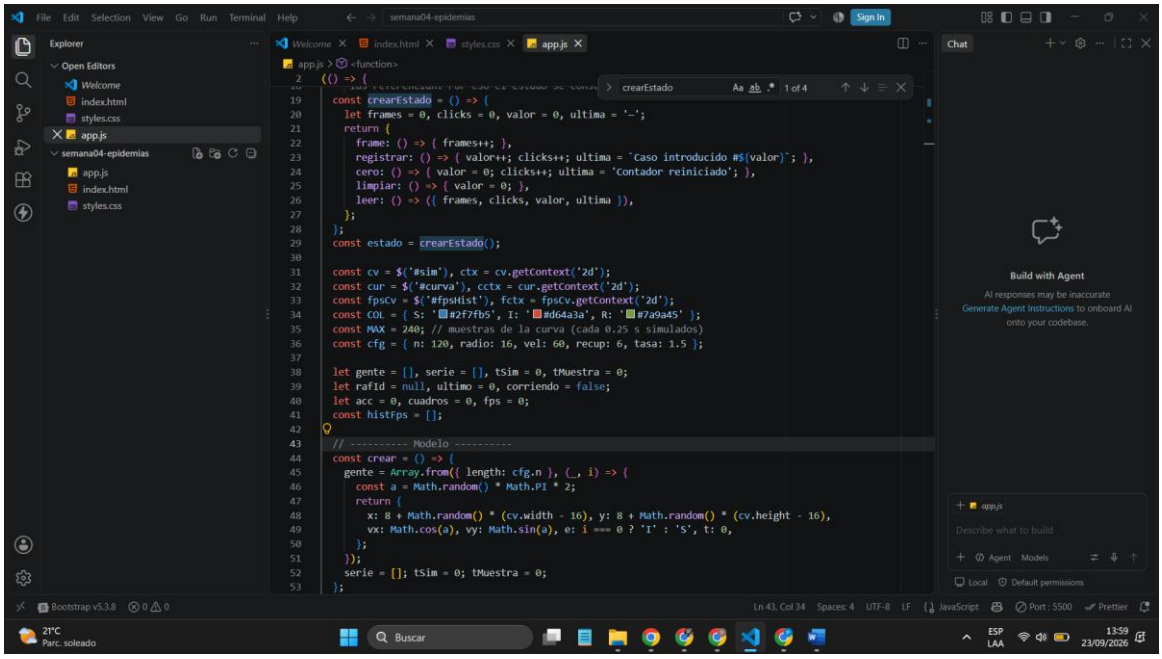
estructura, HTML y canvas

-



-





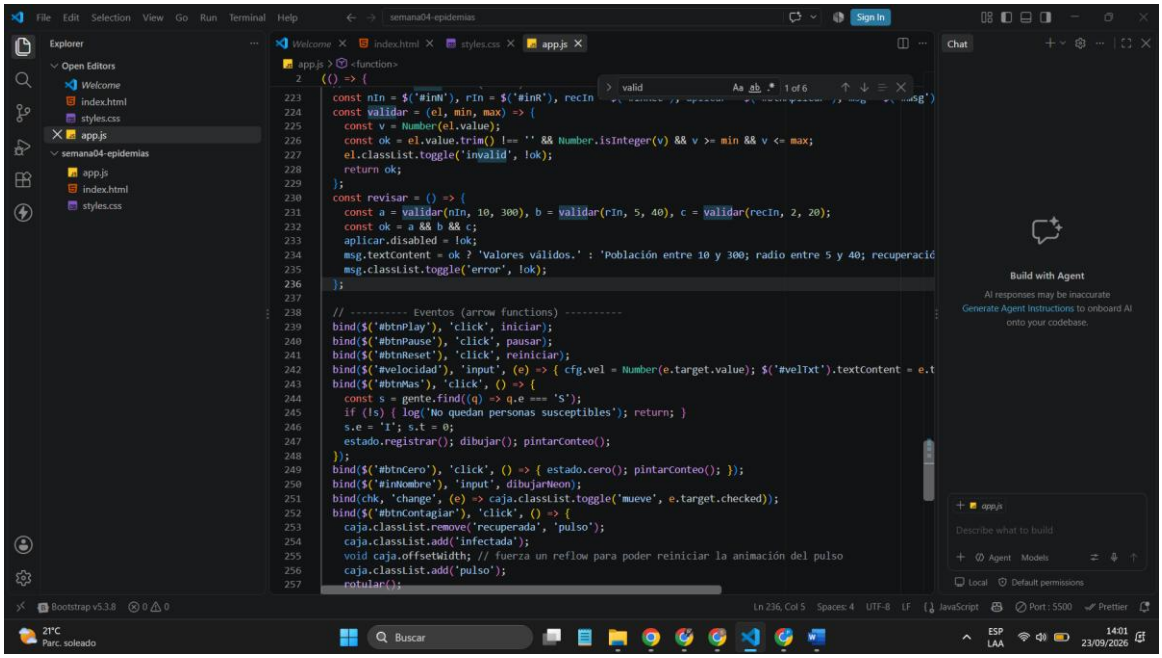
Susceptibles **38** · Infectadas **3** · Recuperadas **79**

2 IIFE, closures y arrow functions

Todo el código vive dentro de una IIFE, así que no crea variables globales. El estado de la simulación está guardado en un closure: sigue disponible entre un frame y otro sin salir del scope. Los handlers son arrow functions, que conservan el `this` léxico. Con el botón puedes introducir un caso: infecta a una persona susceptible y el closure suma uno al contador.

0	1989	0	—
Casos introducidos	Frames simulados	Clics en total	Última acción

Introducir un caso | Reiniciar contador



3 Manipulación del DOM y validación

Esta caja representa a una persona. Con `classList.toggle()` y variables CSS cambia de estado sin usar `style` inline: al contagiarla se pone roja y emite un pulso que representa su radio de contagio; al recuperarla se pone verde; en cuarentena se detiene. Marca "Persona en movimiento" para que se desplace. Más abajo se validan los parámetros de la epidemia.

Infectada

☐ Persona en movimiento | Contagiar | Recuperar | Aislar en cuarentena | Restablecer

Población (10 a 300 personas)

120

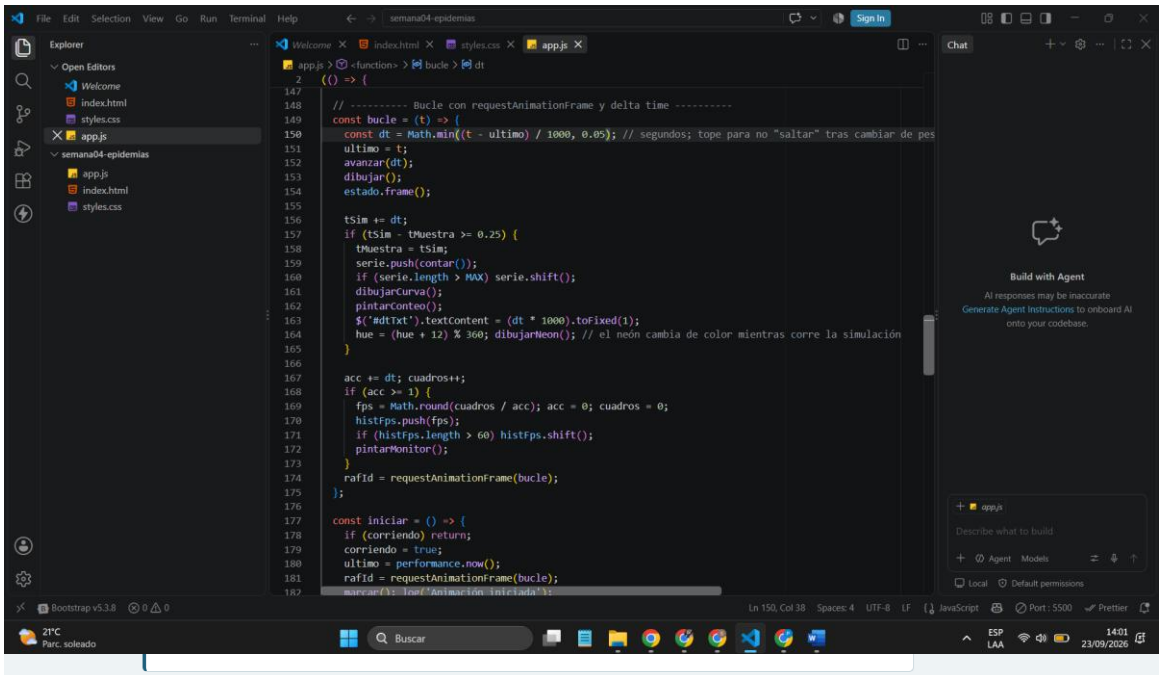
Radio de contagio (5 a 40 px)

16

Tiempo de recuperación (2 a 20 segundos)

6

Aplicar y reiniciar




```
// ---------- Bucle con requestAnimationFrame y delta time ----------
const bucle = (t) => {
  const dt = Math.min((t - ultimo) / 1000, 0.05); // segundos; tope para no "saltar" tras cambiar de pes
  ultimo = t;
  avanzar(dt);
  dibujar();
  estado.frame();

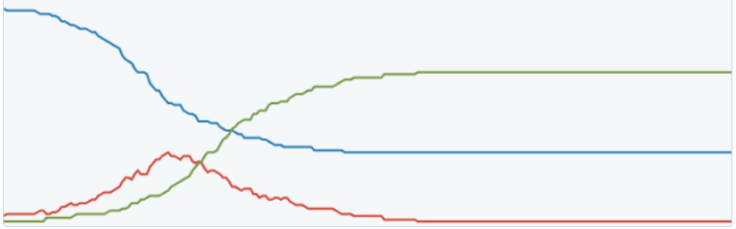
  tSim += dt;
  if (tSim - tMuestra >= 0.25) {
    tMuestra = tSim;
    serie.push(contar());
    if (serie.length > MAX) serie.shift();
    dibujarCurva();
    pintarConteo();
    $("#dtTxt").textContent = (dt * 1000).toFixed(1);
    hue = (hue + 12) % 360; dibujarNeon(); // el neón cambia de color mientras corre la simulación
  }

  acc += dt; cuadros++;
  if (acc >= 1) {
    fps = Math.round(cuadros / acc); acc = 0; cuadros = 0;
    histFps.push(fps);
    if (histFps.length > 60) histFps.shift();
    pintarMonitor();
  }
  rafId = requestAnimationFrame(bucle);
};

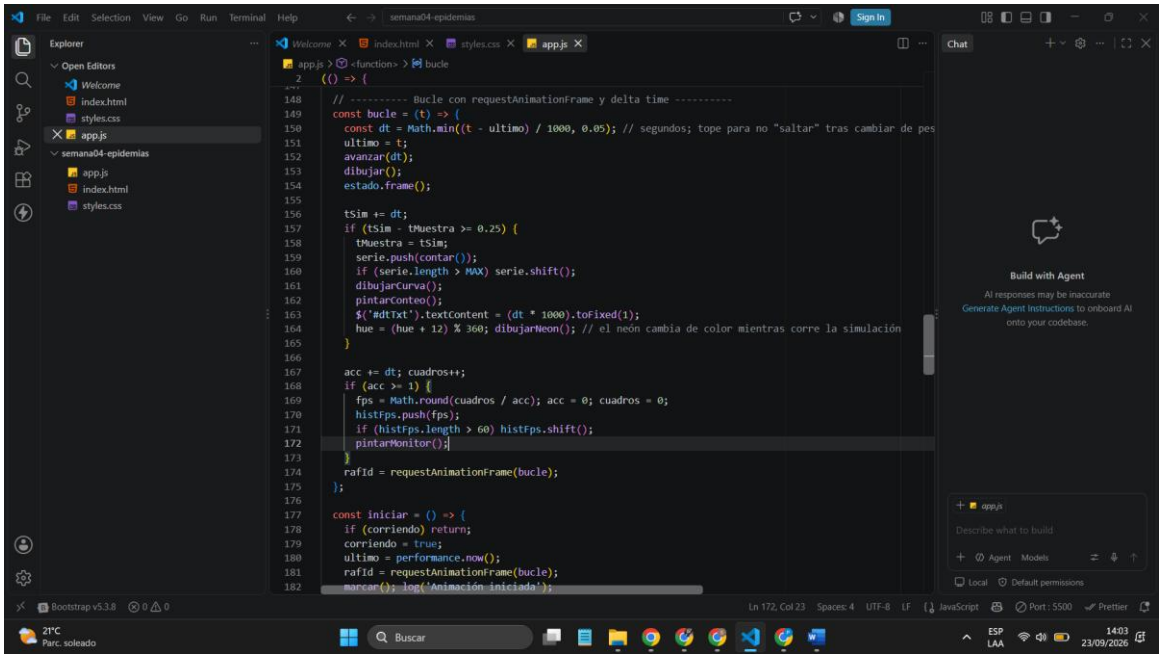
const iniciar = () => {
  if (corriendo) return;
  corriendo = true;
  ultimo = performance.now();
  rafId = requestAnimationFrame(bucle);
  marcar('Animación iniciada');
```

4 Canvas API y requestAnimationFrame

La simulación avanza con `requestAnimationFrame` y delta time, por lo que las personas se mueven igual en pantallas de 60 o 144 Hz. Abajo se dibuja la curva epidémica con `stroke`: la línea roja sube cuando hay más infectados.



Delta time del último cuadro: **16.8** ms





https://github.com/Jackeline-VS/Practica-semana-4--epidemiologia